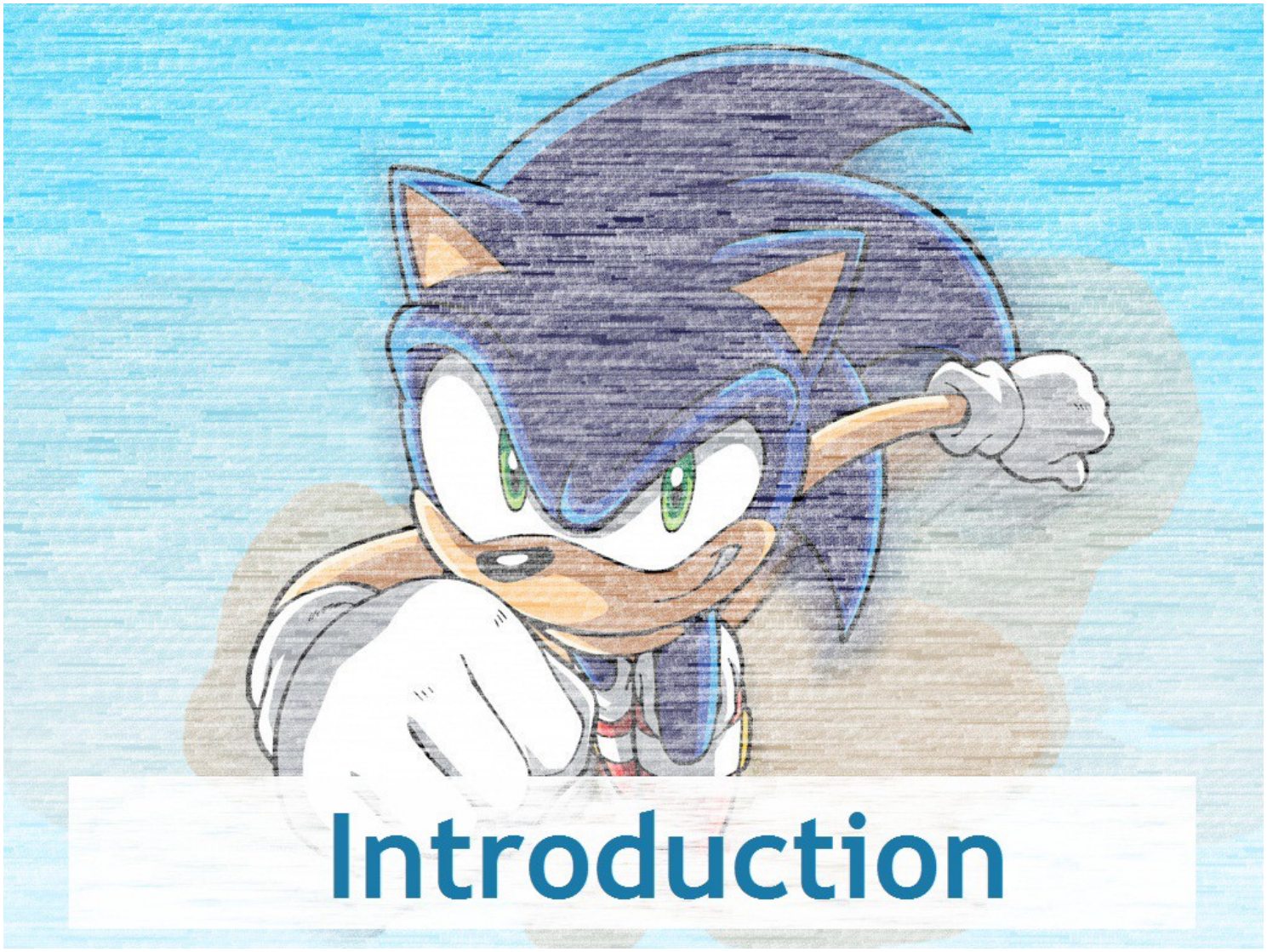
Spring: A Head Start 🔥— Introduction (Part 1)

What, Why, Concepts, and Architecture.

Omar Elgabry Follow
Sep 12, 2017 · 7 min read



Spring: A Head Start 🔥 — Introduction

This is a long series of tutorials. We are going cover:

- An Introduction to Spring framework (what, why, concepts, architecture)
- Beans Configuration (XML, Annotation, Java code)
- Spring Events
- Spring AOP Framework
- Spring MVC
- Spring Data Access (JDBC & Hibernate)
- Building A Web Application
- Spring Gifts 🎁: Security, Logging, Caching, JSON, Excel, AngularJS, etc.

We are about to hit our first steps towards the most popular application development framework for enterprise Java applications.

Hello Spring! 👏

When Spring came out, it was a simpler, a light-weight alternative to J2EE, to make J2EE development easier.

Ok. But, hold on. What enterprise Java refers to?

Enterprise Java refers to Java enterprise software. It's a computer software used to satisfy the needs of an organization rather than individual users. Examples can include: accounting software, billing Management, business process management, CMS, CRM, ERP, etc.

📢 *So, what are the benefits of using Spring?*

- It uses Java **POJOs** (Plain Old Java Object), which make it much simpler to build enterprise-class applications as compared to the heavyweight EJBs (from the earlier

versions of J2EE).

> *A POJO is a Java object not bound by any restriction other than those forced by the Java language specification. So, POJO should not have to extend pre-specified classes in or implement pre-specified interfaces in the framework.*

- Promotes loose coupling by making use of **dependency of injection**. So, instead of hard-coding the dependencies of an object, you simply specify the dependencies via configuration file (or annotations, or java code).
- Uses **AOP (Aspect Oriented Programming).** It allows the separation of cross-cutting concerns (i.e. logging, auditing, declarative transactions, security, caching, etc) from the business logic.
- Minimize the **boilerplate Java code**. It has a collection of helper packages and classes to make it easier and avoid boilerplate code, and you have to worry only about the classes you need and ignore the rest.
- It makes use of some of the **existing technologies** like several ORM frameworks, logging frameworks, JEE, Quartz and JDK timers, and other.
- The core features of the Spring framework can be used in developing any Java application, but there are extensions for building **web applications** on top of the Java EE platform.

Now, we are going to explore some concepts that lie as the ♡ of Spring.

Dependency Injection (DI)

The **Inversion of Control (IoC)** is a design principle in which the flow of control of a system is inverted. It's a general concept, where the dependency injection is one concrete example of it.

> *The Spring IoC Container creates, manages, and injects object's dependencies.*

The dependency injection separates an object from its dependencies, so we can decide which dependencies to be injected during the run time.

> 💡 *This decision is done in Spring in the beans configuration file (or using annotations or java code) instead of modifying the code, as we'll see later.*

```
public class App {
    private Service service;

    // A service object will be injected by Spring IoC container
    public App (Service service) {
        this.service = service;
    }
}
```

Dependency Injection (DI).java hosted with ♡ by GitHub view raw

And the benefit is, we are able to replace and extend the functionality of the dependencies (services) without changing the dependent (App) class at all.

Aspect Oriented Programming (AOP)

It allows the separation of *cross-cutting concerns* from the business logic.

> 💡 *The cross-cutting concerns are features that span across different parts of the application and affects the entire application. Examples → logging, declarative transactions, security, caching, etc.*

📢 *But, why we would use it?*

What happens if we need to change, for example, a specific security consideration across all the application. Such a change would require a major effort, especially, if this security consideration is used across many methods.

So, AOP attempts to solve this problem by allowing us to express cross-cutting concerns in stand-alone modules called "aspects".

For example, a security module "aspect" can include "advice" that performs a security check (actual piece of code that will be executed), and a "join point" which defines when

(i.e. when the code is executed) in the corresponding program the aspect code should be executed.

This can be done either using XML syntax, or syntax based on "AspectJ", which uses Java code.

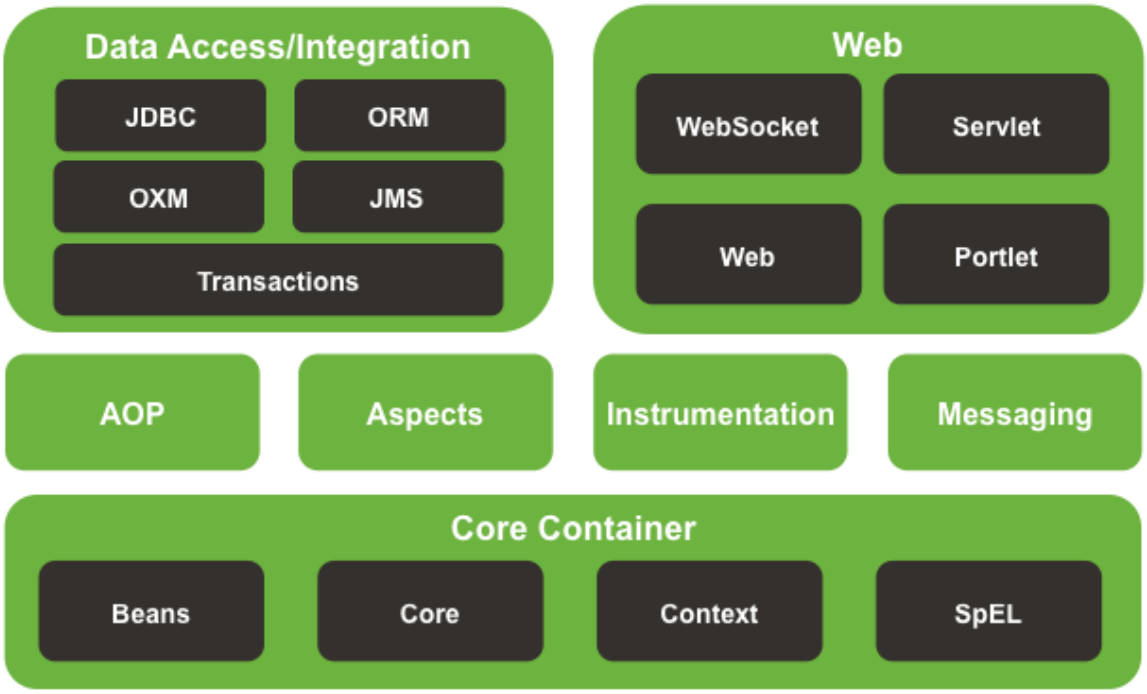
> 📣 ***DI vs AOP*** *→ DI helps you decouple your application objects from it's dependencies, while AOP helps you decouple cross-cutting concerns from the objects.*

Spring Architecture 🏠

Spring is organized in a modular fashion which allows you to worry only about the modules you need and ignore the rest.

> 💡***Modular programming*** *is a software design technique that separates the functionality of a program into independent modules, such that each contains one specific functionality.*
>
> 📣 ***Modularity vs AOP*** *→ The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the aspect.*

It provides about 20 modules. All modules are built on the top of its core container. These modules provide everything that a developer may need for use in the enterprise application development.



Test

Spring Architecture — docs.spring.io

Core Container

It's used for creating beans and manage beans dependencies.

- The **Core** and **Bean** modules provide the fundamental parts of the framework, including the IoC and Dependency Injection features.
- The **Bean** module provides *BeanFactory*, which is a sophisticated implementation of the factory pattern. It decouples the configuration and specification of dependencies from the actual program logic.
- The **Context** module builds on the Core and Beans modules. It is a means to access any objects defined and configured.

> 💡*The context module extends the BeanFactory and adds support for application lifecycle events, and validation. It enables many enterprise services such as JNDI access, EJB integration, remoting and scheduling, and also facilitates easy integration with the other frameworks.*

- The **SpEL** module provides a powerful expression language for querying and manipulating objects during the runtime.

Data Access/Integration Modules

It's used to communicate with the database.

- **JDBC**: Provides a JDBC-abstraction layer that removes the need for JDBC related coding.
- **ORM**: Provides an integration layers for popular object-relational mapping APIs, including JPA, JDO, Hibernate, and iBatis.
- **OXM**: Provides an abstraction layer that supports Object/XML mapping implementations.

- **The Java Messaging Service (JMS)**: Contains features for producing and consuming messages (emails).
- **Transaction**: It supports transaction management for classes.

Web Modules

It's the home of Sring MVC framework. We can integrate other technologies like JSF.

- **Web**: Provides basic web-oriented integration features such as multipart file upload functionality and the initialization of the IoC container using Servlet listeners and a web-oriented application context.
- **Web-MVC**: Contains Spring's Model-View-Controller (MVC) and REST Web Services implementation for web applications.
- **Web-Sockets**: Provides support for WebSocket-based, two-way communication between the client and the server in web applications.
- **Web-Portlet**: Provides the MVC implementation to be used in a portlet environment and mirrors the functionality of Web-MVC module.

AOP and Instrumentation

- **AOP**: Provides an aspect-oriented programming implementation allowing you to separate the cross-cutting concerns from the business logic (as explained before).
- **Aspects**: It's a separate module provides integration with AspectJ.
- **Instrumentation**: Provides support to class instrumentation support and classloader implementations.

Messaging

It serves as a foundation for messaging-based applications.

It brings the features of asynchronous messaging system and provides support for STOMP to be used over WebScokets for users to exchange messages.

Test

It supports the unit testing and integration testing of Spring components with JUnit or TestNG. It also provides mock objects that you can use to test your code in isolation.

Setting Up The Environment 💪

Before digging deeper, and write out first code in Spring, we need to prepare our development environment.

[1] Java Development Kit (JDK)

Make sure that Java Development Kit (JDK) is already installed.

[2] Install Java Application Server.

There are many options. One of the most widely used is Tomcat Server.

So, go to the "Download" section, choose the version, and download the appropriate link from "Binary Distributions".

After download, run the installer, and verify the installation by typing http://localhost:8080/ in the browser URL (by default it listens to 8080 port).

[3] Install Java Integrated Development Environment (IDE).

Again, there are many options to use. Eclipse is the most popular.

So, search for "Eclipse IDE for Java EE Developers", and download it. After it downloads, unzip the file, and run the eclipse application.

[4] Connecting Tomcat to Eclipse

We can use Eclipse to run Tomcat and easily deploy our application to Tomcat. Click on the "Servers" tab in Eclipse, create a new server, and choose Tomcat.

> 💡*Eclipse gives you the option to download and install Tomcat if not already installed when creating a new server.*

[5] Downloading Spring JAR Files / Using Maven

Go to Spring's download page, and choose the latest version. Create a new "Java Project", and a new folder also called "lib" inside the project. Now, copy all the jar files of Spring framework inside the "lib" folder.

Spring needs "Apache Commons Logging" to log its own data. So, download the jar file, and add it to "lib" folder of your java project.

Finally, you need to add these jar files to the build path to be referenced by our Java application. Go to "properties" of your project, choose "Java Build Path", then "Libraries", and add all jar files in "lib" folder.

📢 *But, what about Maven? …*

Maven (Recommended)

It's a tool used to describe how software is built, and its dependencies. It then downloads all the jar files from the internet automatically. So, instead of manually downloading the jar files, we can use Maven instead.

To download, install Maven for Eclipse, here is a step-by-step guide.

After that, we can add all the dependencies in the "pom.xml" file, which can be found inside the Eclipse project as soon as you convert it to Maven project.

Each dependency has properties, like the *groupId*, *artifactId*, and *version*. You can find all dependencies and their properties values in Maven Repository.

Finally, to install these dependencies after adding them to "pom.xml" file, right-click on your project, choose "Maven", then "Update Project". That's it.

Below is a simple example of "pom.xml" file with the dependencies we'll need for working with Spring framework.

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-i
  <!-- ... -->
  <properties>
    <springframework.version>4.3.4.RELEASE</springframework.version>
 </properties>
 <dependencies>
        <!-- Apache Common Logging API -->
        <dependency>
           <groupId>commons-logging</groupId>
           <artifactId>commons-logging</artifactId>
           <version>1.2</version>
        </dependency>
```

```
        <!-- Spring -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-webmvc</artifactId>
            <version>${springframework.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-tx</artifactId>
            <version>${springframework.version}</version>
        </dependency>
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-orm</artifactId>
            <version>${springframework.version}</version>
        </dependency>
  </dependencies>
  <build> <!-- ... --> </build>
 </project>
```

More about Maven can be found on their documentation and Wikipedia.

Thank you for reading! If you enjoyed it, please clap 👏 for it.

Java Spring Framework Programming Software Development Other

About Help Legal

Get the Medium app

Download on the App Store GET IT ON Google Play